



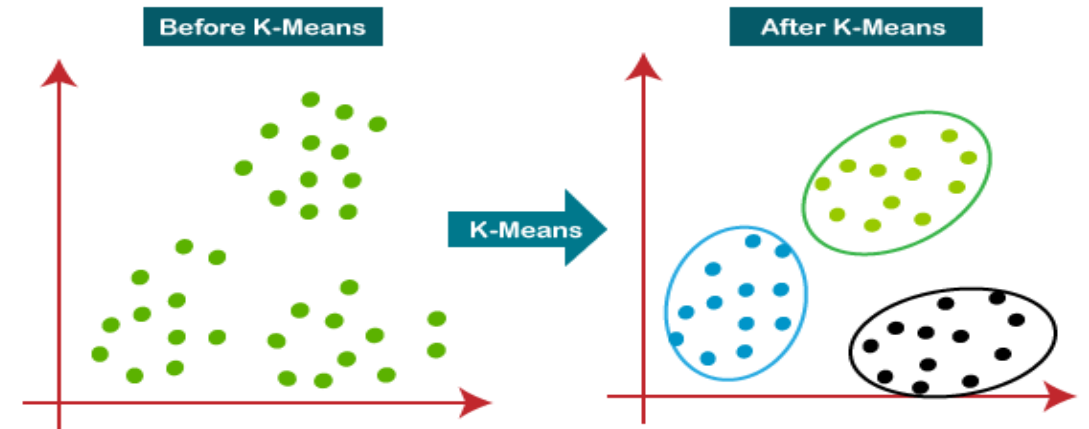
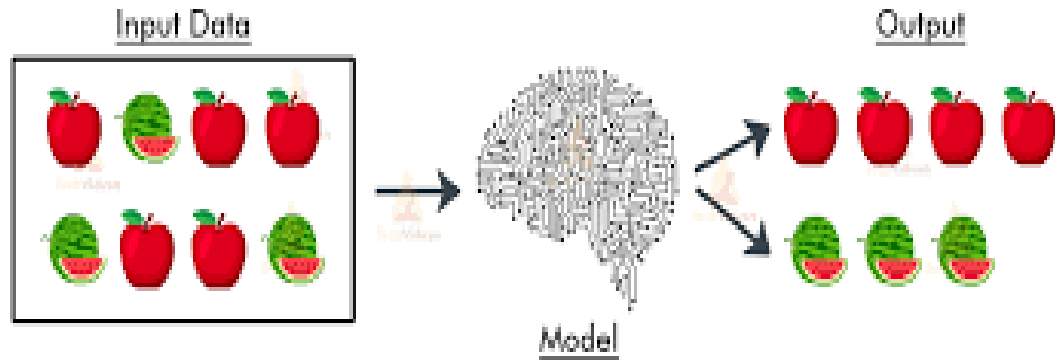
Bahir Dar University

BSc. In Software Engenering

Unsupervised MachineLearning

By:Alemwork M.

Unsupervised Learning in ML



Dimensionality Reduction & Principal Component Analysis

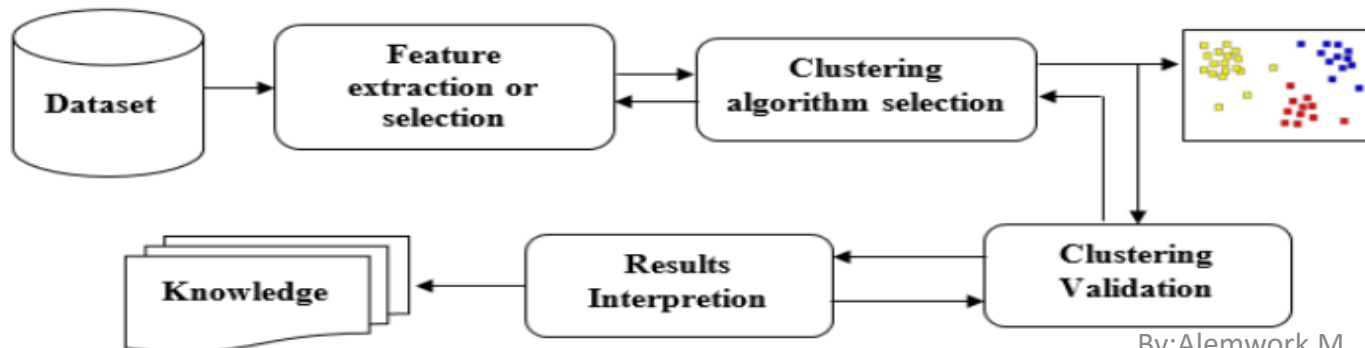
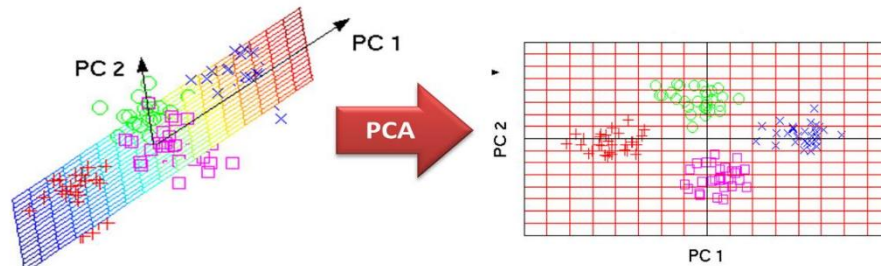
All Features



Feature Selection



Final Features



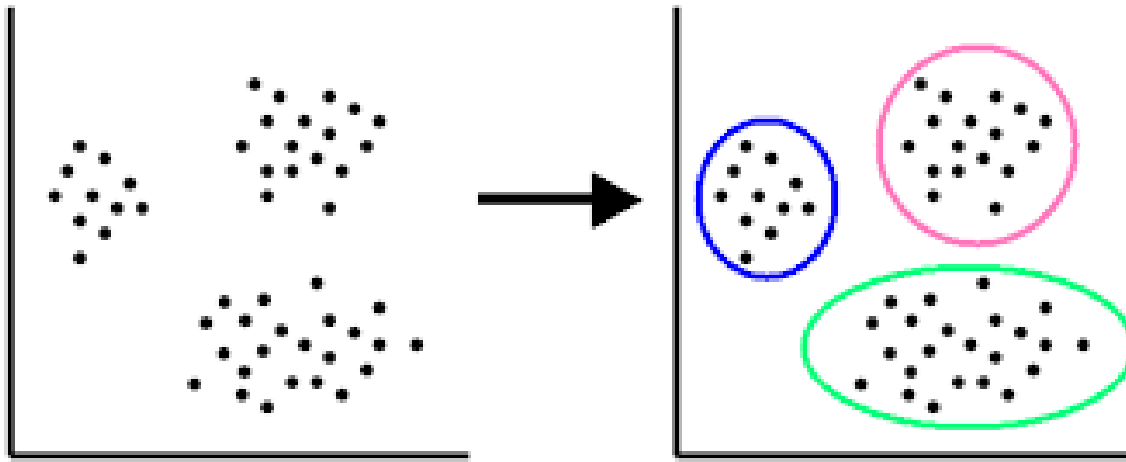
By:Alemwork M.



Unsupervised learning

- Unsupervised learning aims to find the underlying structure or the distribution of data.

We want to explore the data to find some intrinsic structures in them.



- Learn from inputs $x_1, \dots, x_n \in \mathbb{R}^d$ without any labels $y_1, \dots, y_n$.

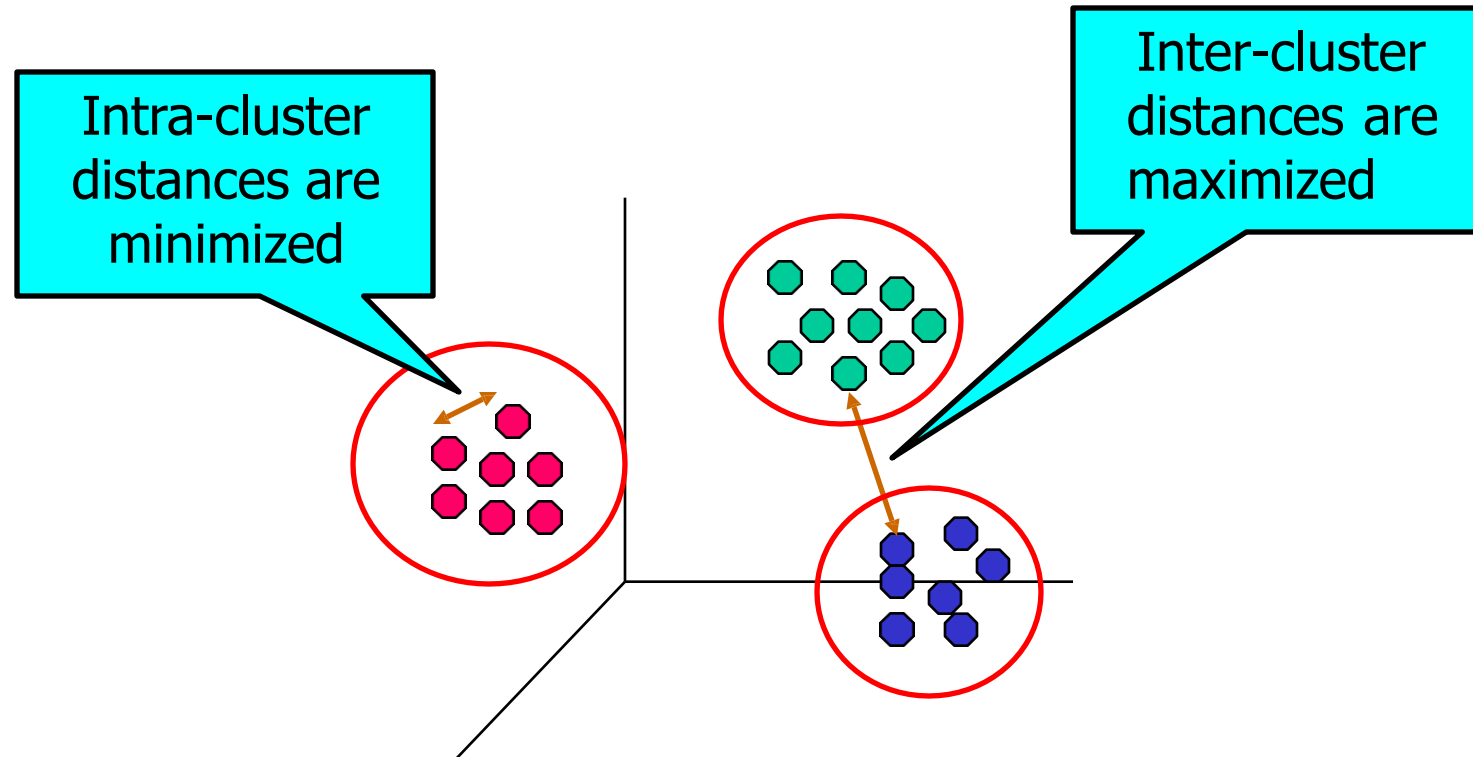
***No predefined classes - Clustering**

What is Cluster Analysis?

- **Cluster:** a collection of data objects
 - Similar to one another within the same cluster
 - Dissimilar to the objects in other clusters
- **Cluster analysis**
 - Finding similarities between data according to the characteristics found in the data and grouping similar data objects into clusters

What is Cluster Analysis?

- Finding groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups



Clustering-Applications

- ▶ **Recommender systems**: organizing products and customers into groups that are similar



- ▶ **Social networks**: cluster users into groups that have similar interests/preferences



- ▶ **WWW document classification**: organize webpages (e.g., news articles) into clusters with similar content (sports, economics, ...)

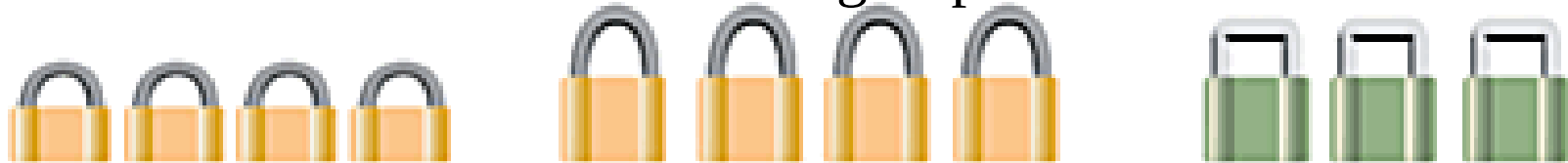


Example: Clustering

- The example below demonstrates the clustering of padlocks of same kind. There are a total of 10 padlocks which various in color, size, shape, etc.



- How many possible clusters of padlocks can be identified?
 - There are three different kind of padlocks; which can be grouped into three different clusters.
 - The padlocks of same kind are clustered into a group as shown below.



Examples of Clustering Applications

- Marketing: Help marketers discover distinct groups in their customer bases, and then use this knowledge to develop targeted marketing programs
- Land use: Identification of areas of similar land use in an earth observation database
- Insurance: Identifying groups of motor insurance policy holders with a high average claim cost
- City-planning: Identifying groups of houses according to their house type, value, and geographical location
- Earth-quake studies: Observed earth quake epicenters should be clustered along continent faults

What is Good Clustering

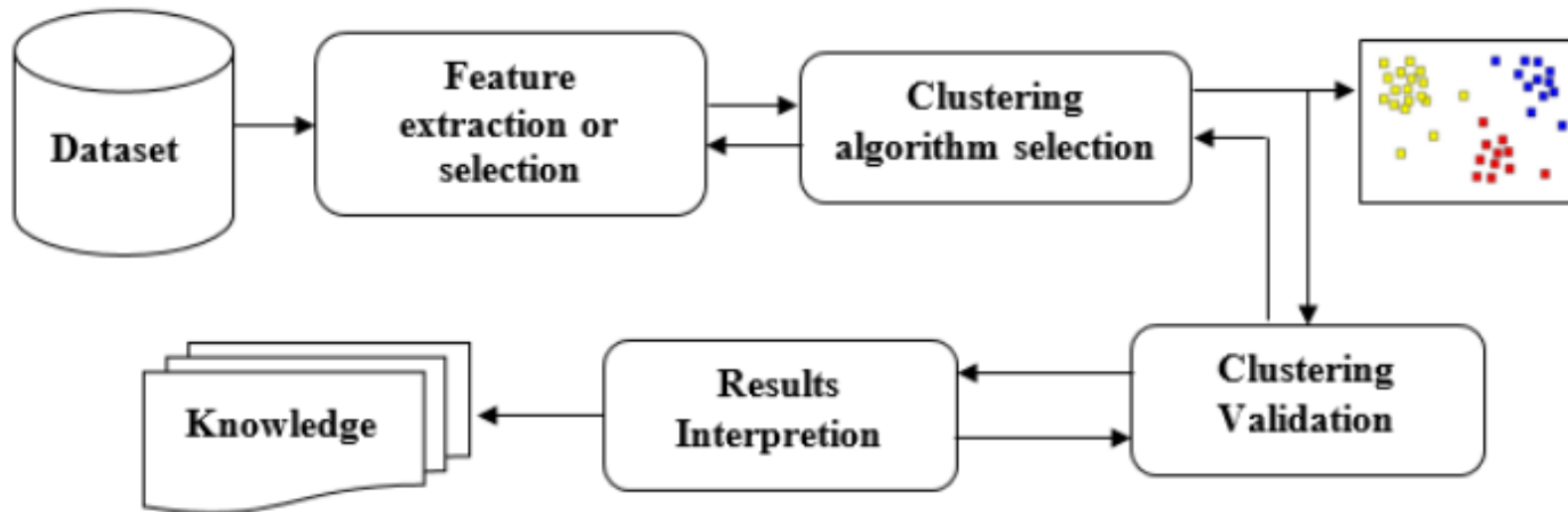
- A good clustering method will produce high quality clusters with
 - high intra-class similarity
 - low inter-class similarity
- The quality of a clustering result depends on both the similarity measure used by the method and its implementation.

Similarity and Dissimilarity Between Objects

- Distances are normally used to measure the similarity or dissimilarity between two data objects
- *Distance measure greatly depends on the type of the attribute value (binary, nominal, ordinal, etc) that each object will assume.*

Procedures of Cluster Analysis

- A typical cluster analysis consists of four steps



Overview of clustering

- Feature Selection

- identifying the most effective subset of the original features to use in clustering

- Feature Extraction

- transformations of the input features to produce new salient features.

- Inter-pattern Similarity

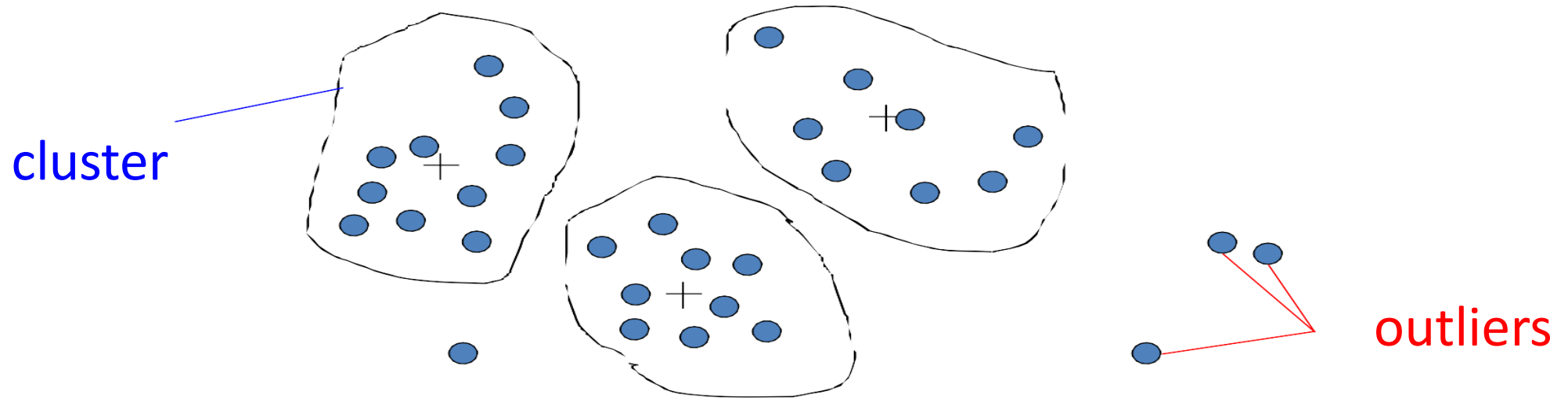
- measured by a distance function defined on pairs of patterns.

- Grouping

- methods to group similar patterns in the same cluster

Outliers

- **Outliers** are objects that do not belong to any cluster or form clusters of very small cardinality



- In some applications we are interested in discovering outliers, not clusters (**outlier analysis**)

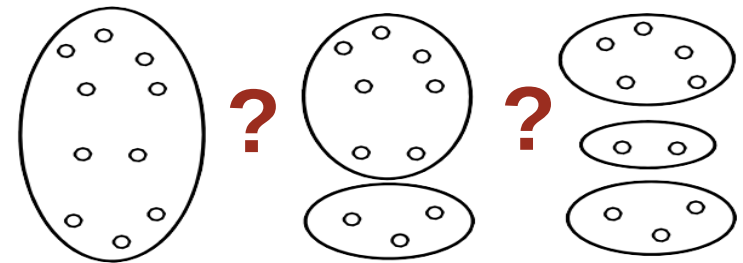
Evaluation

- *Intra-cluster cohesion* (compactness):
 - Cohesion measures how near the data points in a cluster are to the cluster centroid.
 - Sum of squared error (SSE) is a commonly used measure.
- *Inter-cluster separation* (isolation):
 - Separation means that different cluster centroids should be far away from one another.
- In most applications, **expert judgments** are still the key.

Measuring clustering validity

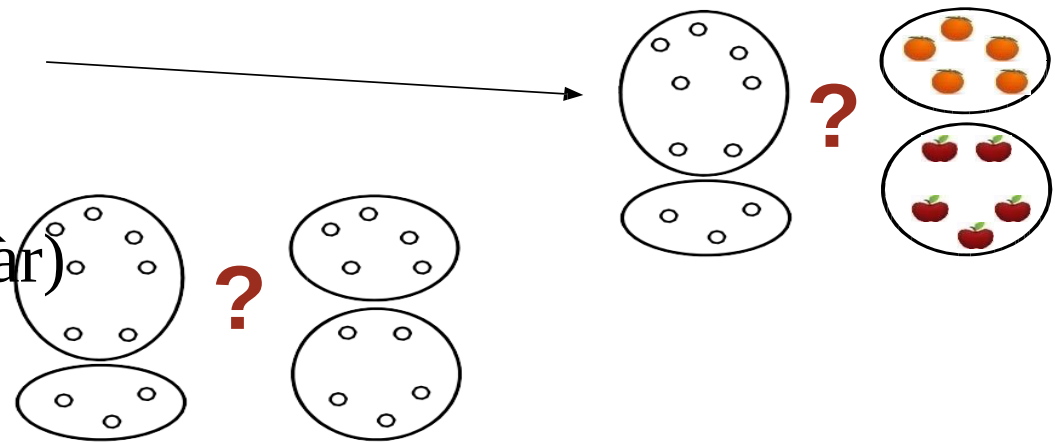
Internal Index:

- Validate *without* external info
- With different number of clusters
- Solve the number of clusters



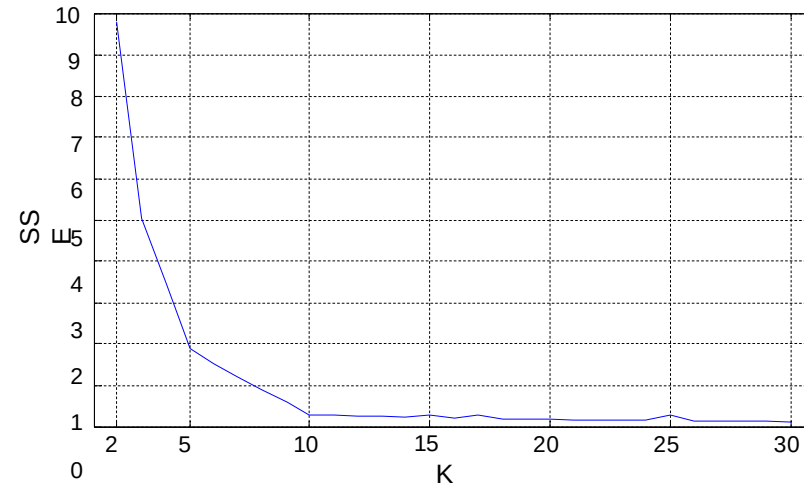
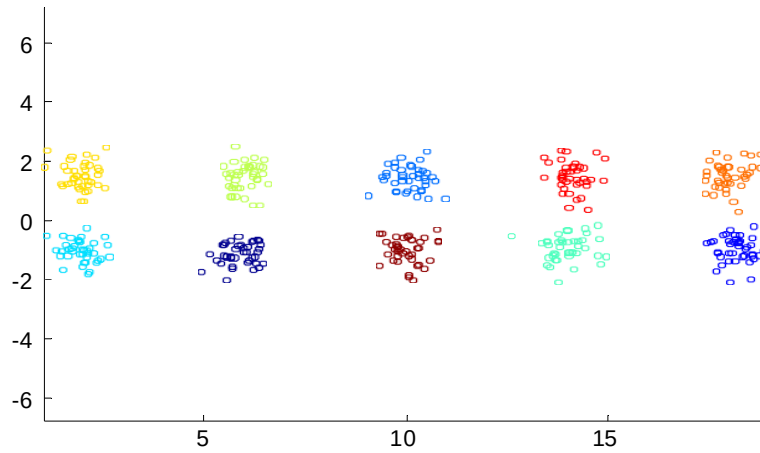
External Index (Reality)

- Validate against ground truth
- Compare two clusters: (how similar)



Internal Measures: SSE

- **Internal Index:** Used to measure the goodness of a clustering structure without reference to external information
 - Example: **SSE**
- SSE is good for comparing two clustering or two clusters (average SSE).
- Can also be used to estimate the number of clusters

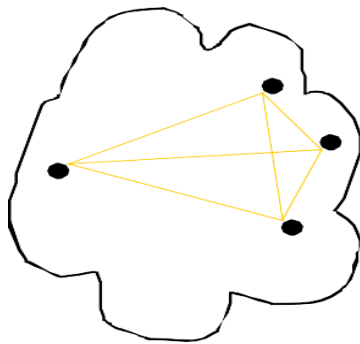


Internal Measures: Cohesion and Separation

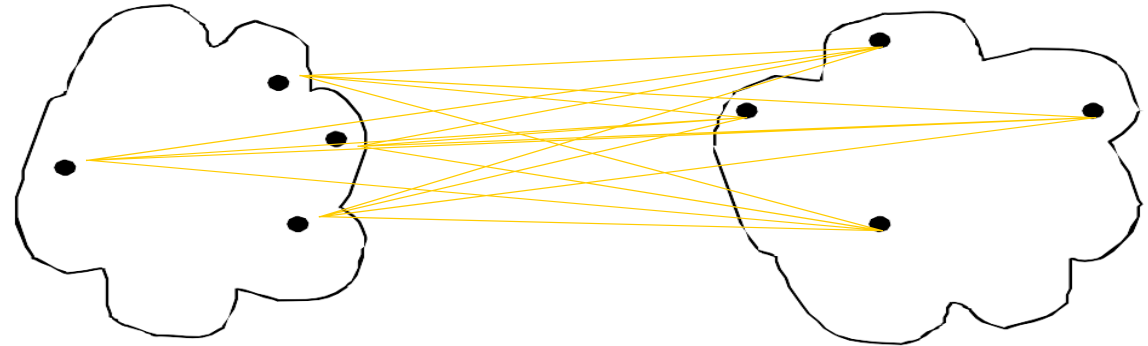
- **Cluster Cohesion:** Measures how closely related are objects in a cluster
- **Cluster Separation:** Measure how distinct or well-separated a cluster is from other clusters

Internal Measures: Cohesion and Separation

- A proximity graph based approach can also be used for cohesion and separation.
 - **Cluster cohesion** is the sum of the weight of all links within a cluster.
 - **Cluster separation** is the sum of the weights between nodes in the cluster and nodes outside the cluster.

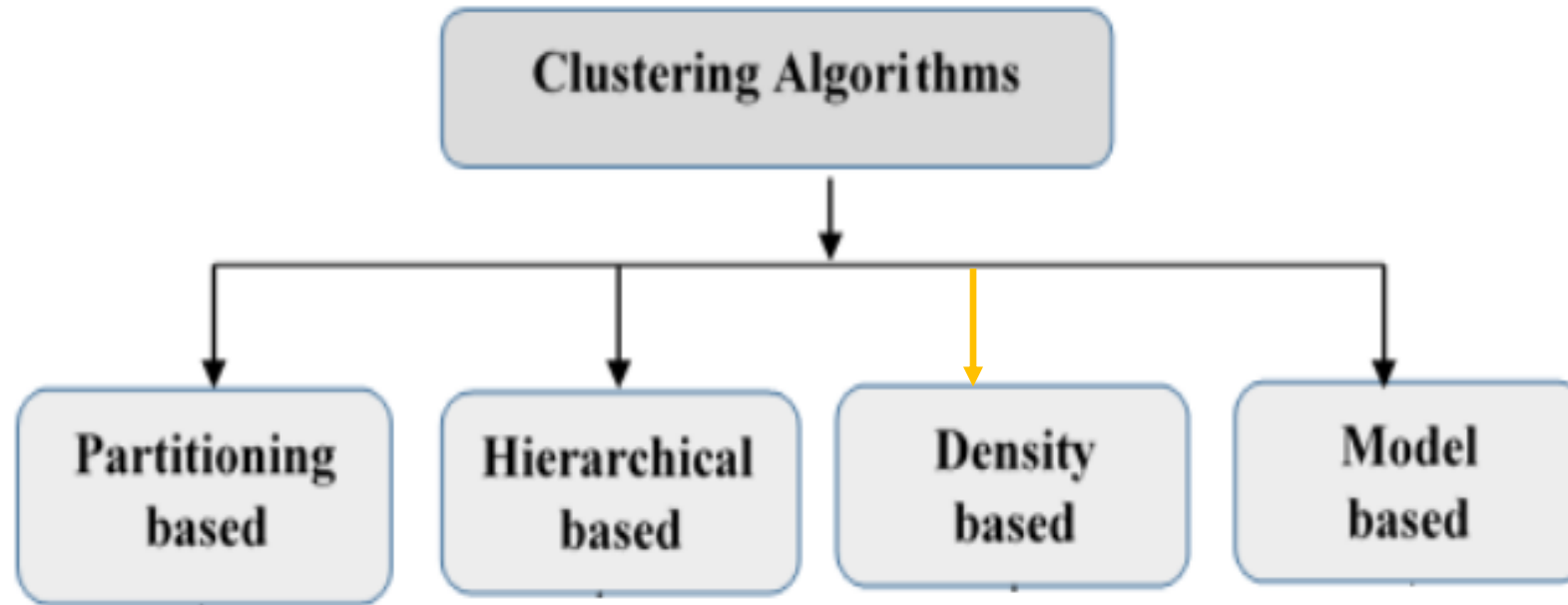


cohesion



separation

Categories of clustering Algorithms



Partitioning method

- Partitioning method: Construct a partition of a database **D** of **n** objects into a set of **k** clusters
- Various approaches have been proposed and some of them are:
 - **K-mean** Approach
 - K-medoid Approach, CLARA (Custering LARge Application) and CLARANS (CLustering Algorithm based on RANdomized Search)

K-means clustering

- K-means is a **partitional clustering** algorithm

- Let the set of data points (or instances) D be

$$\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\},$$

where $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ir})$ is a **vector** in a real-valued space $X \in R^r$, and r is the number of attributes (dimensions) in the data.

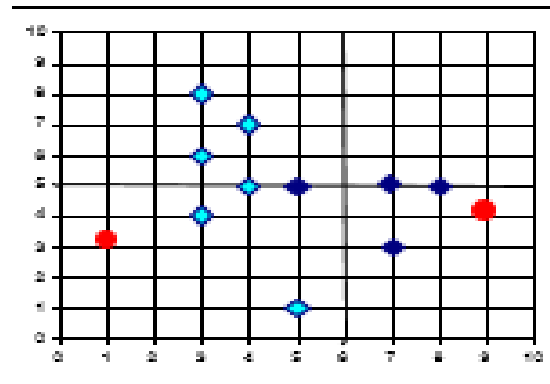
- The k -means algorithm partitions the given data into k clusters.
 - Each cluster has a cluster **center**, called **centroid**.
 - k is specified by the user

K-means algorithm

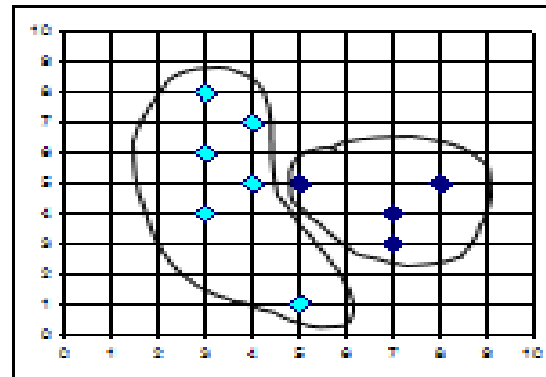
- Given k , the *k-means* algorithm is implemented in 4 steps:
 1. Select K centroid (Can be K values randomly, or K data points randomly)
 2. Partition objects into k subsets. An object will be clustered into class J if it has the smallest distance with this class mean compared to the distance with the other class mean
 3. Compute the new centroids of the clusters of the current partition. The centroid of the j th cluster is the center (mean point) of the data point whose cluster index is found to be the center of class j in the above step.
 4. Go back to Step 3, stop when the process converge.

The *K*-Means Clustering Method

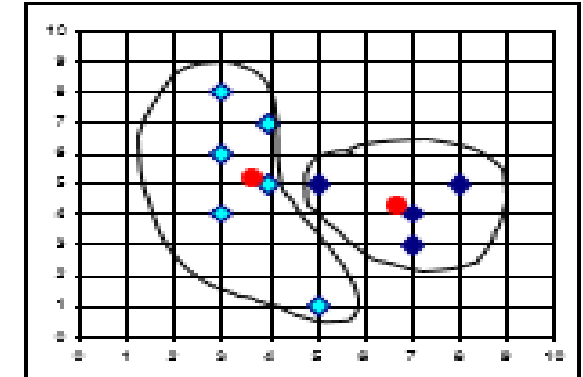
- Example



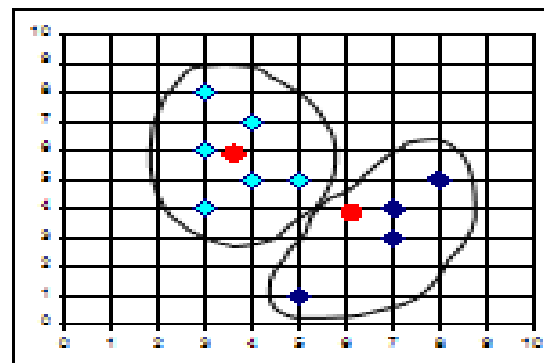
Assign
each
objects
to most
similar
center



Update
the
cluster
means

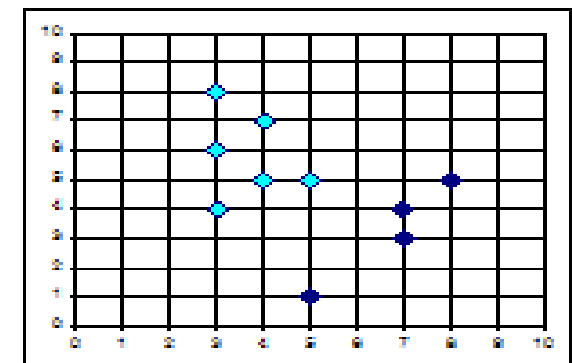


↑ reassign



Update
the
cluster
means

↓ reassign



$K=2$

Arbitrarily
choose K object
as initial cluster
center

Strengths of k-means

- Strengths:

- Simple: easy to understand and to implement
- Efficient: Time complexity: $O(tkn)$, where n is the number of data points, k is the number of clusters, and t is the number of iterations.
- Since both k and t are small. k -means is considered a linear algorithm.
- K-means is the most popular clustering algorithm.

Weaknesses of k-means

- The algorithm is only applicable if the mean is defined.
 - For categorical data, **k-mode** - the centroid is represented by most frequent values.
- The user needs to specify k .
- The algorithm is sensitive to outliers
 - Outliers are data points that are very far away from other data points.
 - Outliers could be errors in the data recording or some special data points with very different values.

Example Problem

- Cluster the following eight points (with (x, y) representing locations) into three clusters :

A1(2, 10) **A2**(2, 5) **A3**(8, 4) **A4**(5, 8)
A5(7, 5) **A6**(6, 4) **A7**(1, 2) **A8**(4, 9).

- Assume that the randomly selected initial cluster centers are:

A1(2, 10), **A4**(8,4) and **A7**(1, 2).

- The distance function between two points $A_j=(x1, y1)$ and $C_i=(x2, y2)$ is defined as: **Manhattan**

$$\text{dis}(A_j, C_i) = |x2 - x1| + |y2 - y1| .$$

- Use k-means algorithm to find optimal centroids to group the given data into three clusters.

Iteration 1

- Starting from point **A1**(2, 10) calculate the distance to each of the three means, by using the distance function:
$$\text{dis}(A1, \text{mean1}) = |2 - 2| + |10 - 10| = 0 + 0 = 0$$
$$\text{dis}(A1, \text{mean2}) = |8 - 2| + |4 - 10| = 6 + 6 = 12$$
$$\text{dis}(A1, \text{mean3}) = |1 - 2| + |2 - 10| = 1 + 8 = 9$$
 - Fill these values in the table & decide which cluster should the point (2, 10) be placed in? The one, where the point has the shortest distance to the mean – i.e. mean 1 (cluster 1), since the distance is 0.
- Next go to the second point **A2**(2, 5) and calculate the distance:
$$\text{dis}(A2, \text{mean1}) = |2 - 2| + |10 - 5| = 0 + 5 = 5$$
$$\text{dis}(A2, \text{mean2}) = |8 - 2| + |4 - 5| = 6 + 1 = 7$$
$$\text{dis}(A2, \text{mean3}) = |1 - 2| + |2 - 5| = 1 + 3 = 4$$
 - So, we fill in these values in the table and assign the point (2, 5) to cluster 3 since mean 3 is the shortest distance from A2.
- Analogically, we fill in the rest of the table, and place each point in one of the clusters

Iteration 1

- First we list all points in the first column of the table below. The initial cluster centers - centroids, are (2, 10), (8,4) and (1, 2) - chosen randomly.
- The Table shows the distance of each data points (instances) from the chosen centroids. The last column shows the cluster the instance should be assigned to based on its distance from the centroids.

Data Points		Cluster 1 with centroid (2,10)	Cluster 2 with centroid (8, 4)	Cluster 3 with centroid (1, 2)	Cluster
A1	(2, 10)	0	12	9	1
A2	(2, 5)	5	7	4	3
A3	(8, 4)	12	0	9	2
A4	(5, 8)	5	7	10	1
A5	(7, 5)	10	2	9	2
A6	(6, 4)	10	2	7	2
A7	(1, 2)	9	9	0	3
A8	(4, 9)	3	9	10	1

Iteration 1

- Next, we need to re-compute the new cluster centers. We do so, by taking the mean of all points in each cluster.
- For Cluster 1, we have three points (A1, A4, A8) and needs to take average of them as new centroid, i.e.
 $((2+5+4)/3, (10+8+9)/3) = (3.67, 9) \rightarrow \text{Cluster 1}$
- For Cluster 2, we have three points (A3, A5, A6). The new centroid is:
 $((8+7+6)/3, (4+5+4)/3) = (7, 4.33) \rightarrow \text{Cluster 2}$
- For Cluster 3, we have two points (A2, A7). The new centroid is: $((2+1)/2, (5+2)/2) = (1.5, 3.5) \rightarrow \text{Cluster 3}$
- Since centroids changes in Iteration1, we go to the next Iteration (epoch2) using the new means we computed.
- ***The iteration continues until the centroids do not change anymore.***

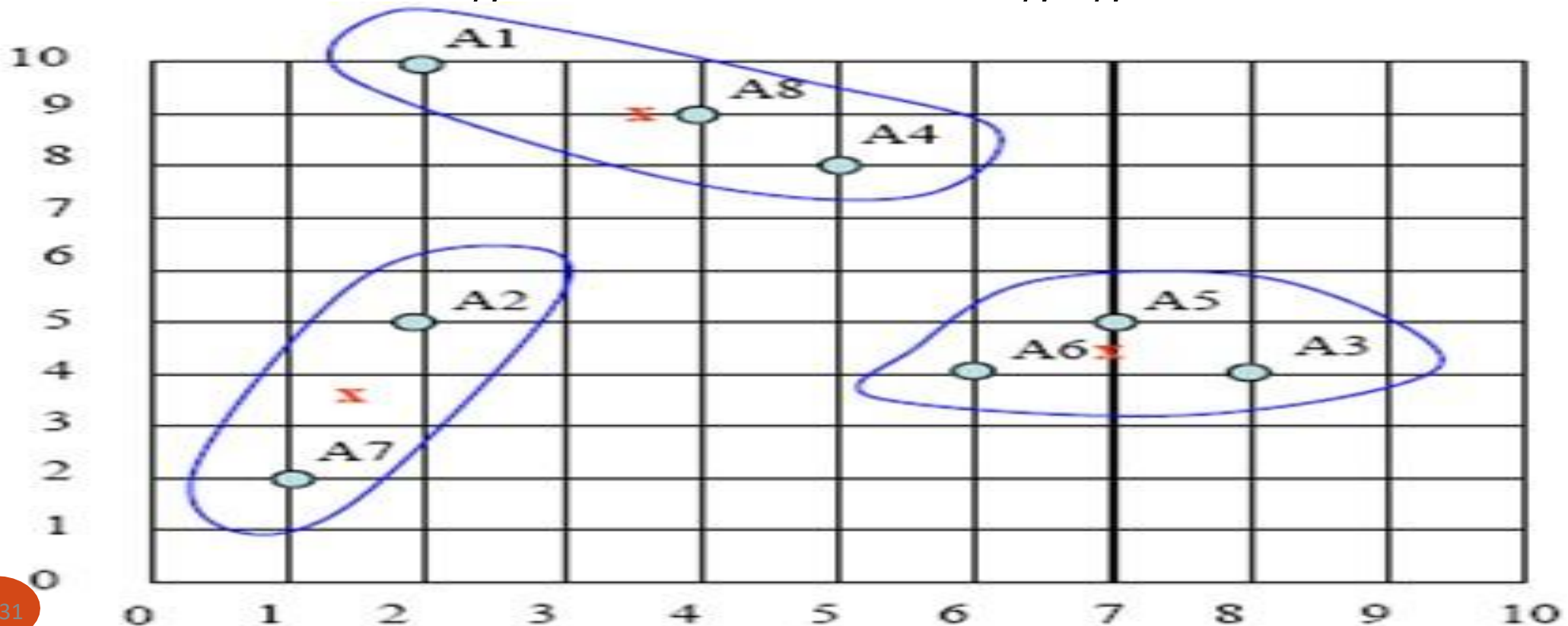
Second epoch

- After the 2nd epoch the results would be:
cluster 1: {A1,A4,A8} with new centroid=(3.67,9); cluster 2:
{A3,A5,A6} with new centroid = (7,4.33); cluster 3:
{A2,A7} with new centroid=(1.5,3.5)
- Using the new centroid compute cluster members again.

Data Points		Cluster 1 with centroid (3.67, 9)	Cluster 2 with centroid (7, 4.33)	Cluster 1 with centroid (1.5, 3.5)	Cluster
A1	(2, 10)				
A2	(2, 5)				
A3	(8, 4)				
A4	(5, 8)				
A5	(7, 5)				
A6	(6, 4)				
A7	(1, 2)				
A8	(4, 9)				

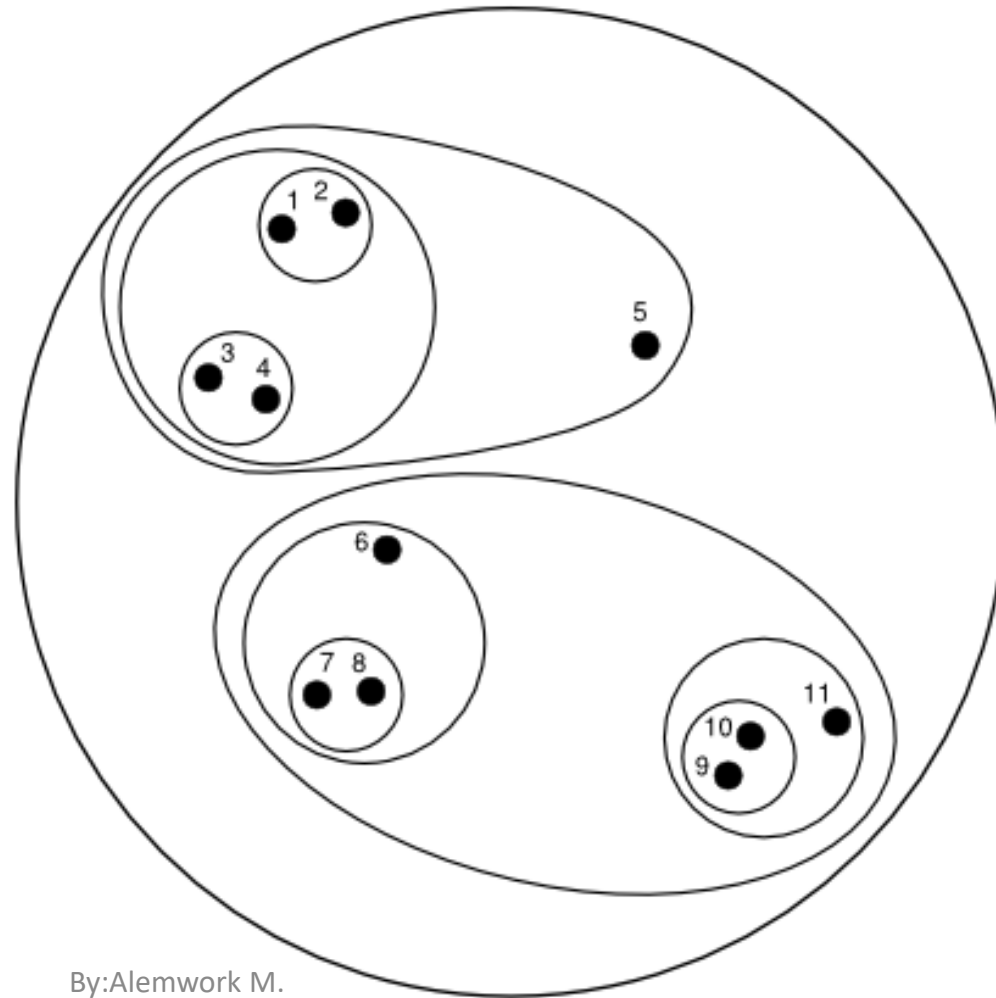
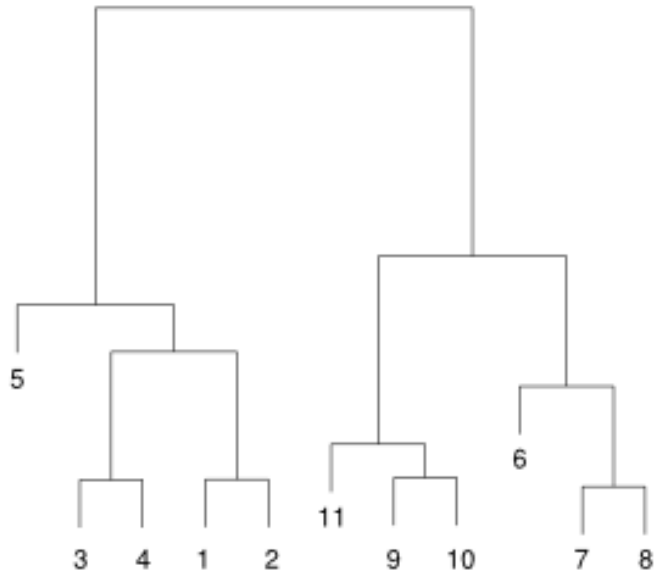
Final results

- Finally in the 2th epoch there is no change of members of clusters and centroids. So the algorithm stops.
- The result of clustering is shown in the following figure

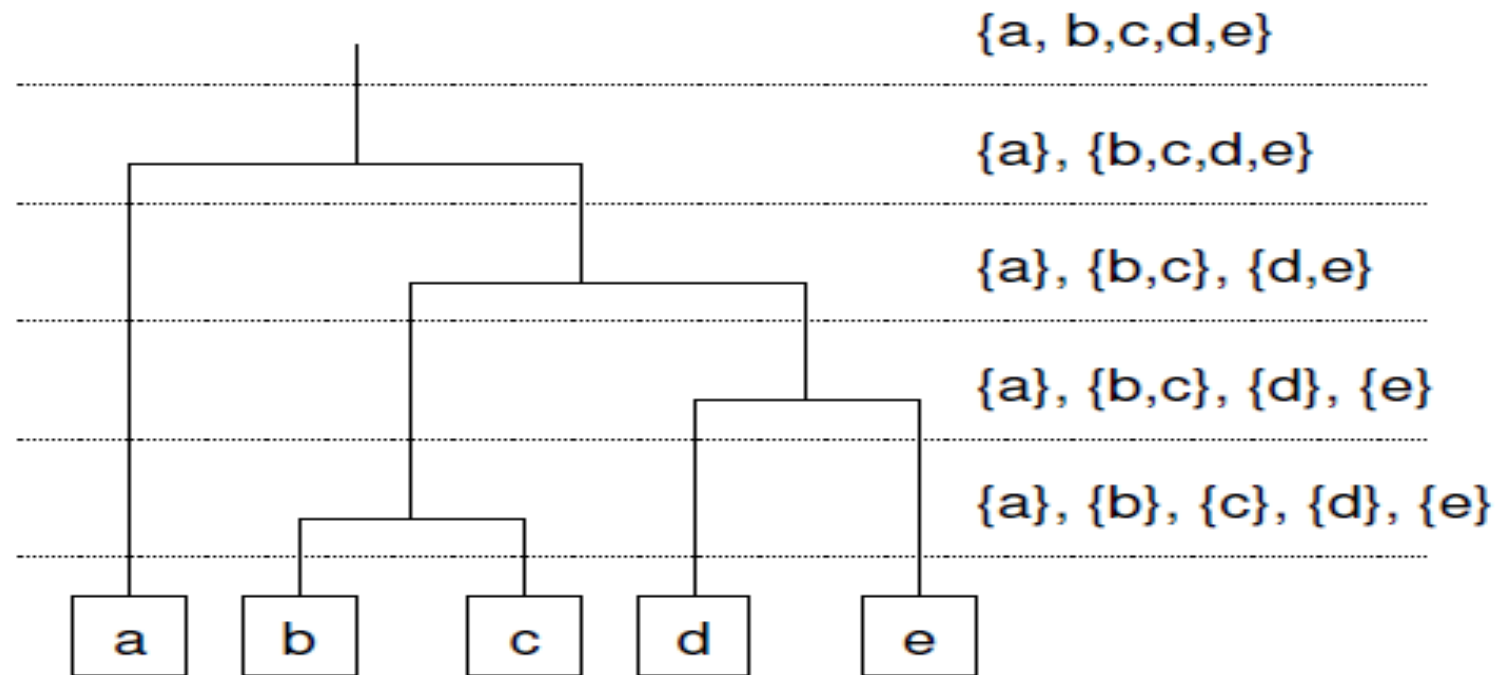


Hierarchical Clustering

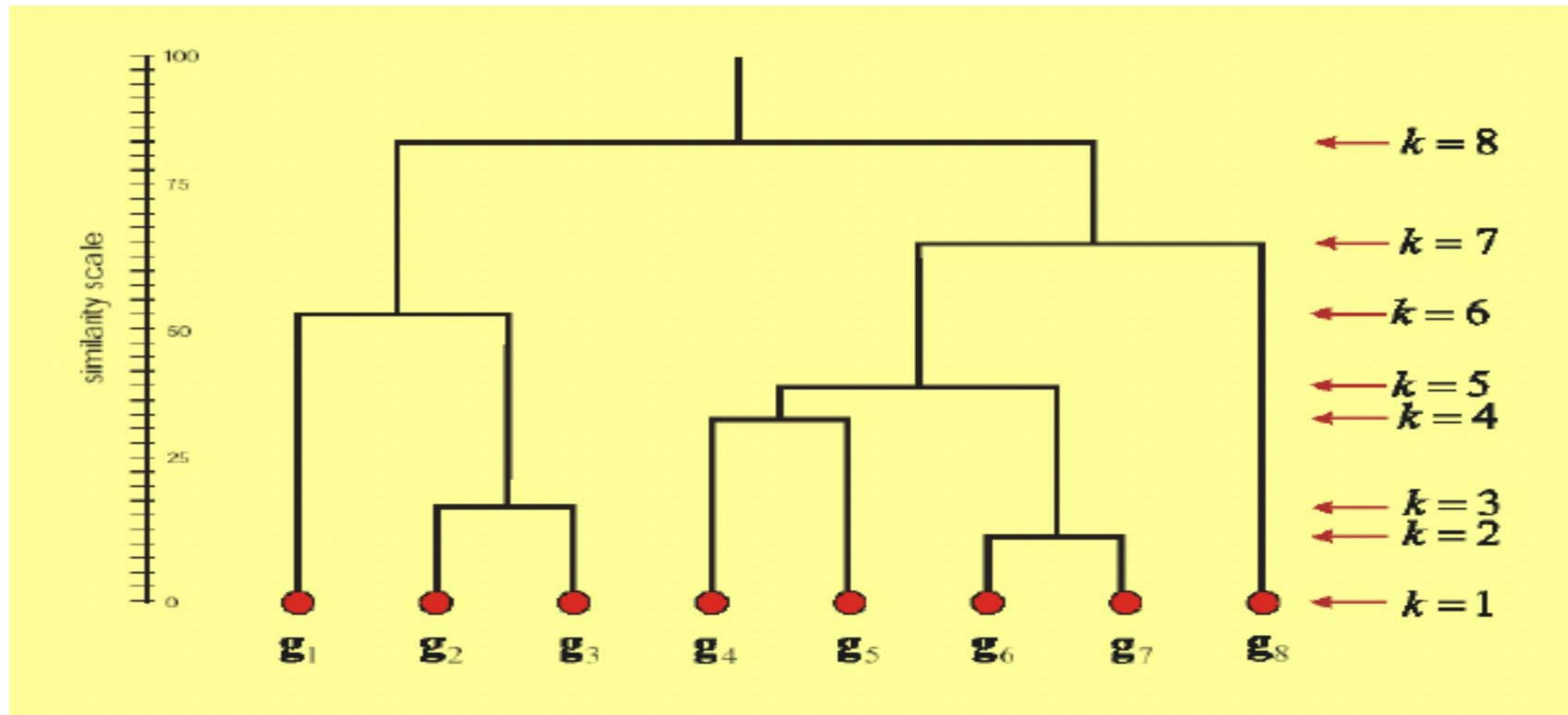
- Produce a nested sequence of clusters, a **tree**, also called **Dendrogram**.



Example of hierarchical clustering



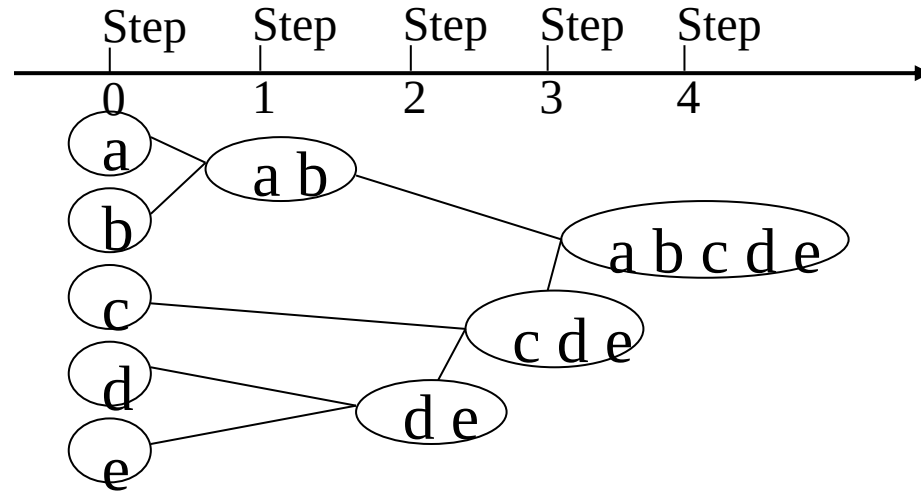
Hierarchical clustering



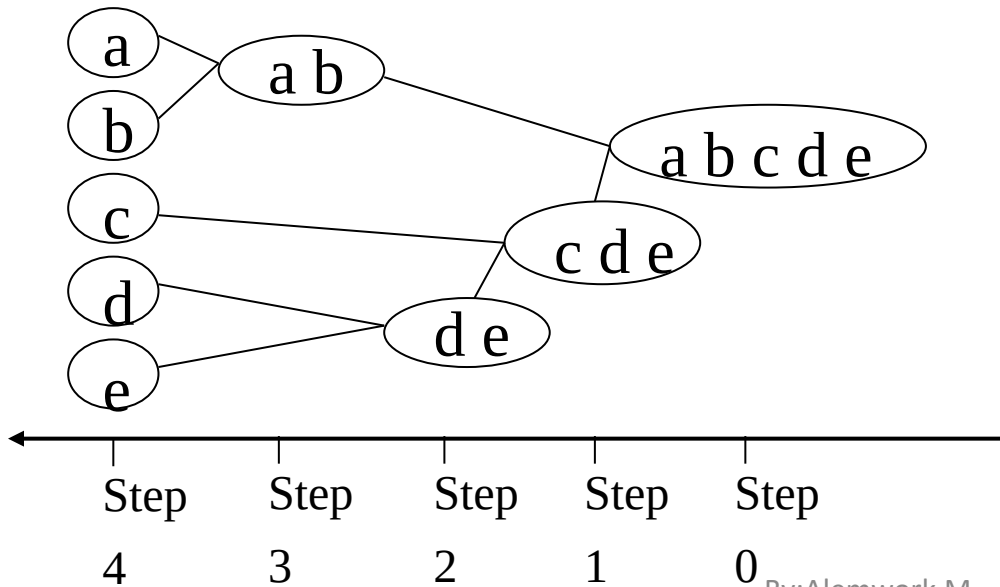
Types of hierarchical clustering

- **Agglomerative (bottom up) clustering:** It builds the dendrogram (tree) from the bottom level, and
 - merges the most similar (or nearest) pair of clusters
 - stops when all the data points are merged into a single cluster (i.e., the root cluster).
- **Divisive (top down) clustering:** It starts with all data points in one cluster, the root.
 - Splits the root into a set of child clusters. Each child cluster is recursively divided further
 - stops when only singleton clusters of individual data points remain, i.e., each cluster with only a single point

Types of hierarchical clustering



**Agglomerative Nesting
(AGNES)**



**Divisive Analysis
(DIANA)**

Agglomerative clustering

It is more popular than divisive methods.

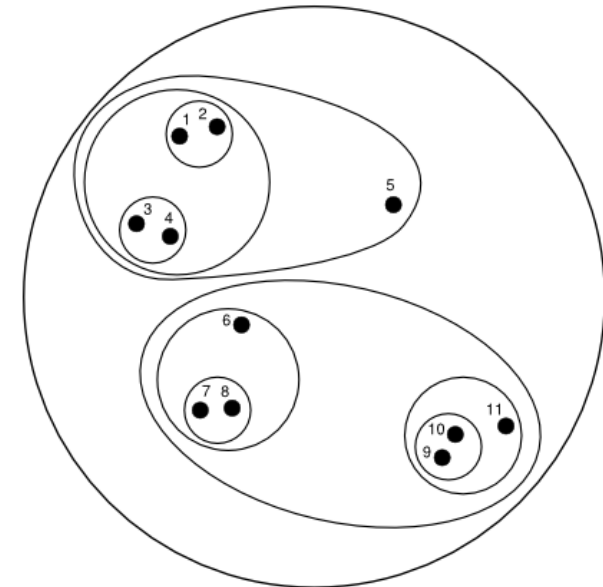
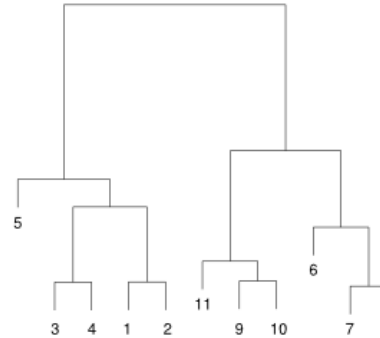
- At the beginning, each data point forms a cluster (also called a node).
- Merge nodes/clusters that have the least distance.
- Go on merging
- Eventually all nodes belong to one cluster

Agglomerative clustering algorithm

```

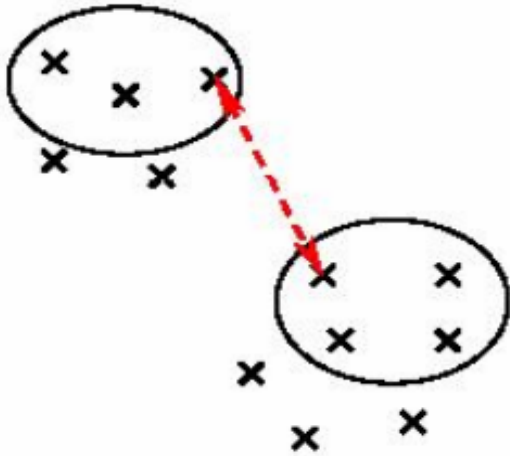
1: function HIERARCHICALCLUSTERING(inputs  $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ )
2:   assign each input to a cluster
3:   repeat
4:     link the two clusters with minimal distance
5:   until only a single root cluster left
6:   return tree of cluster linkages
7: end function

```

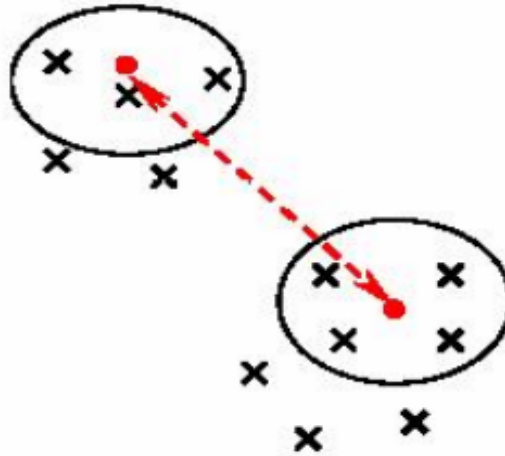


Measuring the distance of two clusters

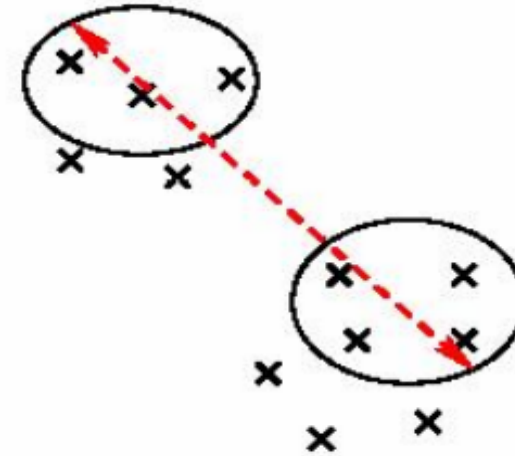
- Simple linkage



- Average linkage



- Complete linkage



- ▶ Let $S_j \subseteq \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ be the set of inputs contained in the j th cluster
- ▶ **Simple linkage:** $d(i, j) := \min_{\mathbf{x} \in S_i, \tilde{\mathbf{x}} \in S_j} \|\mathbf{x} - \tilde{\mathbf{x}}\|$
- ▶ **Average linkage:** $d(i, j) := \text{mean}_{\mathbf{x} \in S_i, \tilde{\mathbf{x}} \in S_j} \|\mathbf{x} - \tilde{\mathbf{x}}\|$
- ▶ **Complete linkage:** $d(i, j) := \max_{\mathbf{x} \in S_i, \tilde{\mathbf{x}} \in S_j} \|\mathbf{x} - \tilde{\mathbf{x}}\|$

Hierarchical Clustering: Problems and Limitations

- **Once a decision is made to combine two clusters, it cannot be undone**
- **No objective function is directly minimized**
- **Different schemes have problems with one or more of the following:**
 - ▶ Sensitivity to noise and outliers
 - ▶ Difficulty handling different sized clusters and convex shapes
 - ▶ Breaking large clusters

Dimensionality reduction techniques

- Another unsupervised learning setting, e.g PCA (Principal component analysis)
- Dimensionality reduction is simply, the process of reducing the dimension of your feature set.
- Your feature set could be a dataset with a hundred columns (i.e. features).
 - High-Dimensions = Lot of Features

Document classification

Features per document =
thousands of words/unigrams
millions of bigrams, contextual
information



Dimensionality reduction techniques

Dimensionality reduction

- Represent the data $x_1, \dots, x_n \in \mathbb{R}^d$ in a subspace of lower dimension with as little loss of information as possible
- **Advantages:**
 - Visualization
 - Lower computation and time complexity
 - Avoid overfitting and reduce noise

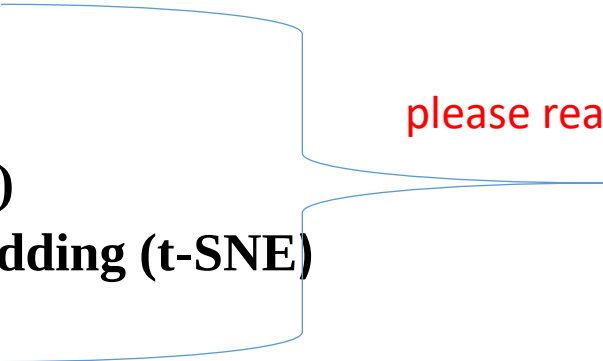
Dimensionality reduction techniques

- **Problem settings:**

- Given $x_1, \dots, x_n \in \mathbb{R}^d$
- find a k -dimensional subspace
- such that the data projected onto that space
- is as close to the original data as possible

- **Common techniques:**

- **Principal Component Analysis(PCA)**
- **Factor Analysis**
- **Linear Discriminant Analysis(LDA)**
- **Multiple Correspondence Analysis (MCA)**
- **t-Distributed Stochastic Neighbour Embedding (t-SNE)**



please read the details of each

Dimensionality reduction techniques

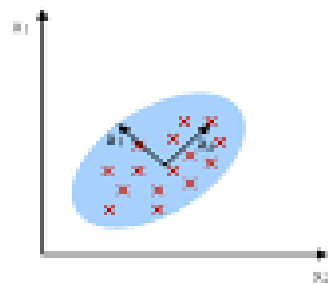
- The most common and well known dimensionality reduction (PCA, LDA, FA)
 - **Principal Component Analysis(PCA):** PCA rotates and projects data along the direction of increasing variance. The features with the maximum variance are the principal components.

```
from numpy import array
from numpy import mean
from numpy import cov
from numpy.linalg import eig
# define a matrix
A = array([[1, 2], [3, 4], [5, 6]])
print(A)
# calculate the mean of each column
M = mean(A.T, axis=1)
print(M)
# center columns by subtracting column means
C = A - M
print(C)
# calculate covariance matrix of centered matrix
V = cov(C.T)
print(V)
# eigendecomposition of covariance matrix
values, vectors = eig(V)
print(vectors)
print(values)
# project data
P = vectors.T.dot(C.T)
print(P.T)
```

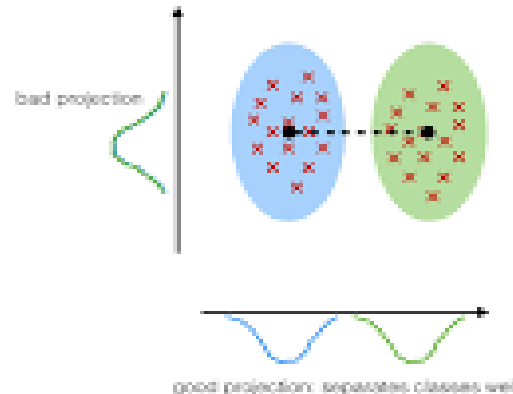
Dimensionality reduction techniques

- **Linear Discriminant Analysis(LDA):** projects data in a way that the class separability is maximized. Examples from same class are put closely together by the projection. Examples from different classes are placed far apart by the projection.

PCA:
component axes that
maximize the variance



LDA:
maximizing the component
axes for class-separation

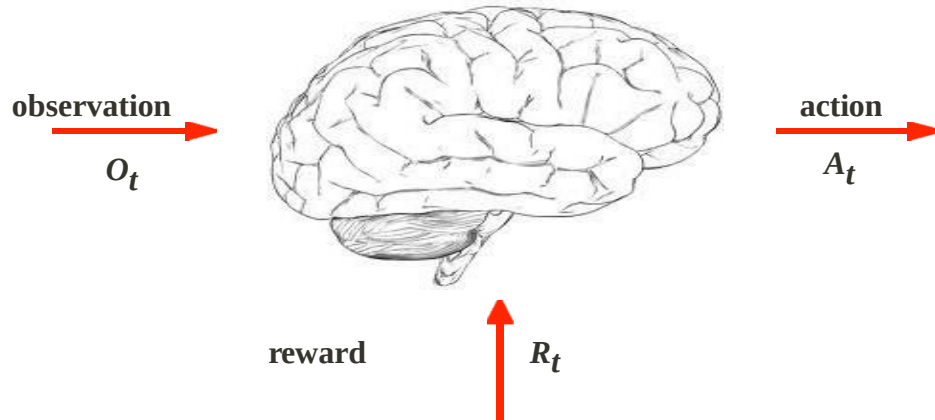


Dimensionality reduction techniques

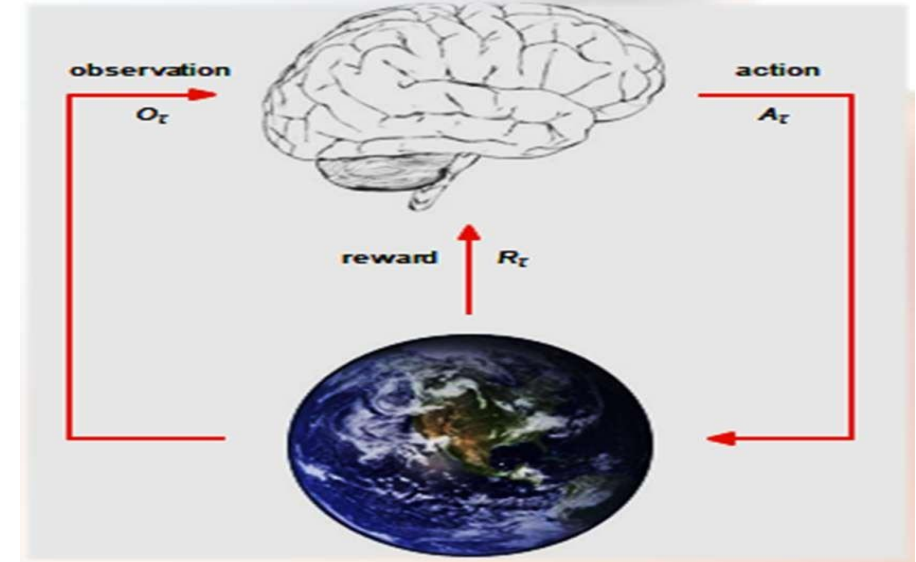
- **Factor Analysis:** a technique that is used to reduce a large number of variables into fewer numbers of factors.
 - The values of observed data are expressed as functions of a number of possible causes in order to find which are the most important.
 - The observations are assumed to be caused by a linear transformation of lower dimensional latent factors and added Gaussian noise.

The phenomenon of reinforcement learning

- **No knowledge of environment**
 - Can only act in the world and observe states and reward



- At each step t the agent:
 - Executes action A_t
 - Receives observation O_t
 - Receives scalar reward R_t
- The environment:
 - Receives action A_t
 - Emits observation O_{t+1}
 - Emits scalar reward R_{t+1}
- t increments at env. step



The phenomenon of reinforcement learning

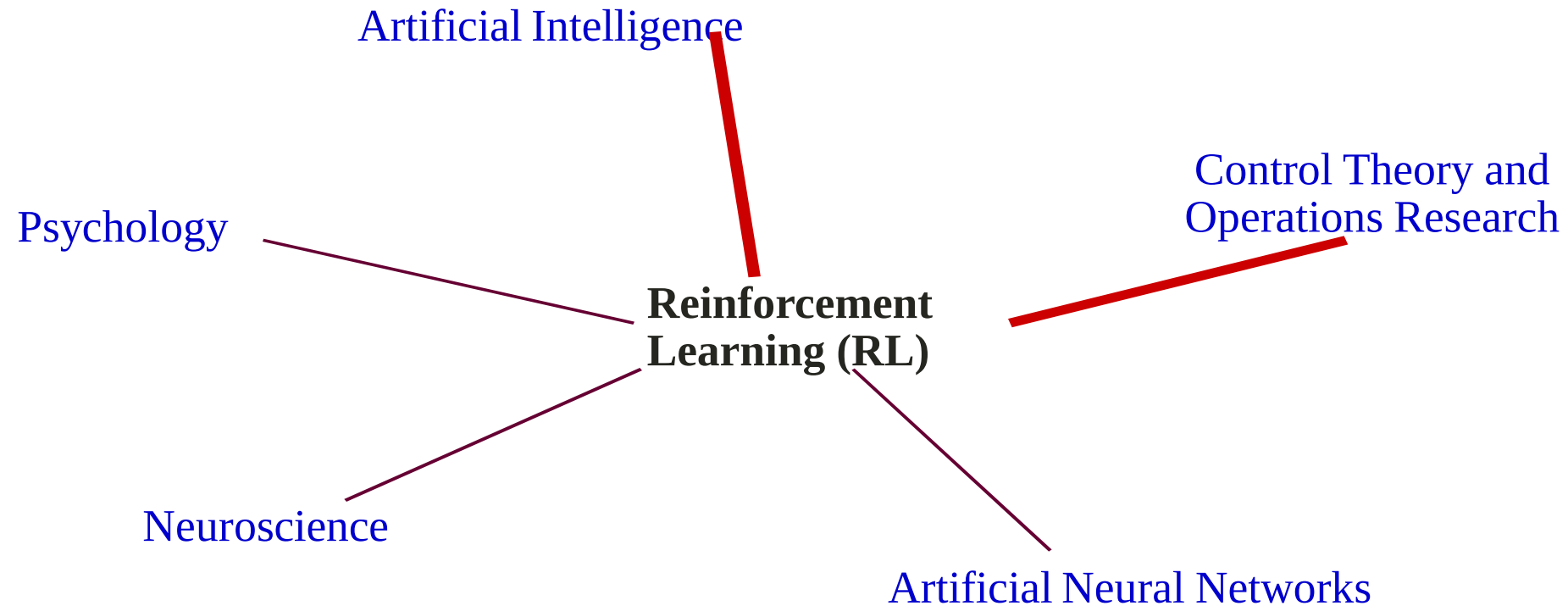
- It is the sequence of observations, actions, rewards
- $H_t = O_1, R_1, A_1, \dots, A_{t-1}, O_t, R_t$
- i.e. all observable variables up to time t
- i.e. the sensorimotor stream of a robot or embodied agent
- What happens next depends on the history:
 - The agent selects actions
 - The environment selects observations/rewards
- State is the information used to determine what happens next
- Formally, state is a function of the history:

The phenomenon of reinforcement learning

- **Many factors make RL difficult:**
 - Actions have non-deterministic effects
 - Which are initially unknown
 - Rewards / punishments are infrequent
 - Often at the end of long sequences of actions
 - How do we determine what action(s) were really responsible for reward or punishment?
 - World is large and complex
- **Nevertheless learner must decide what actions to take**
 - We will assume the world behaves as an MDP

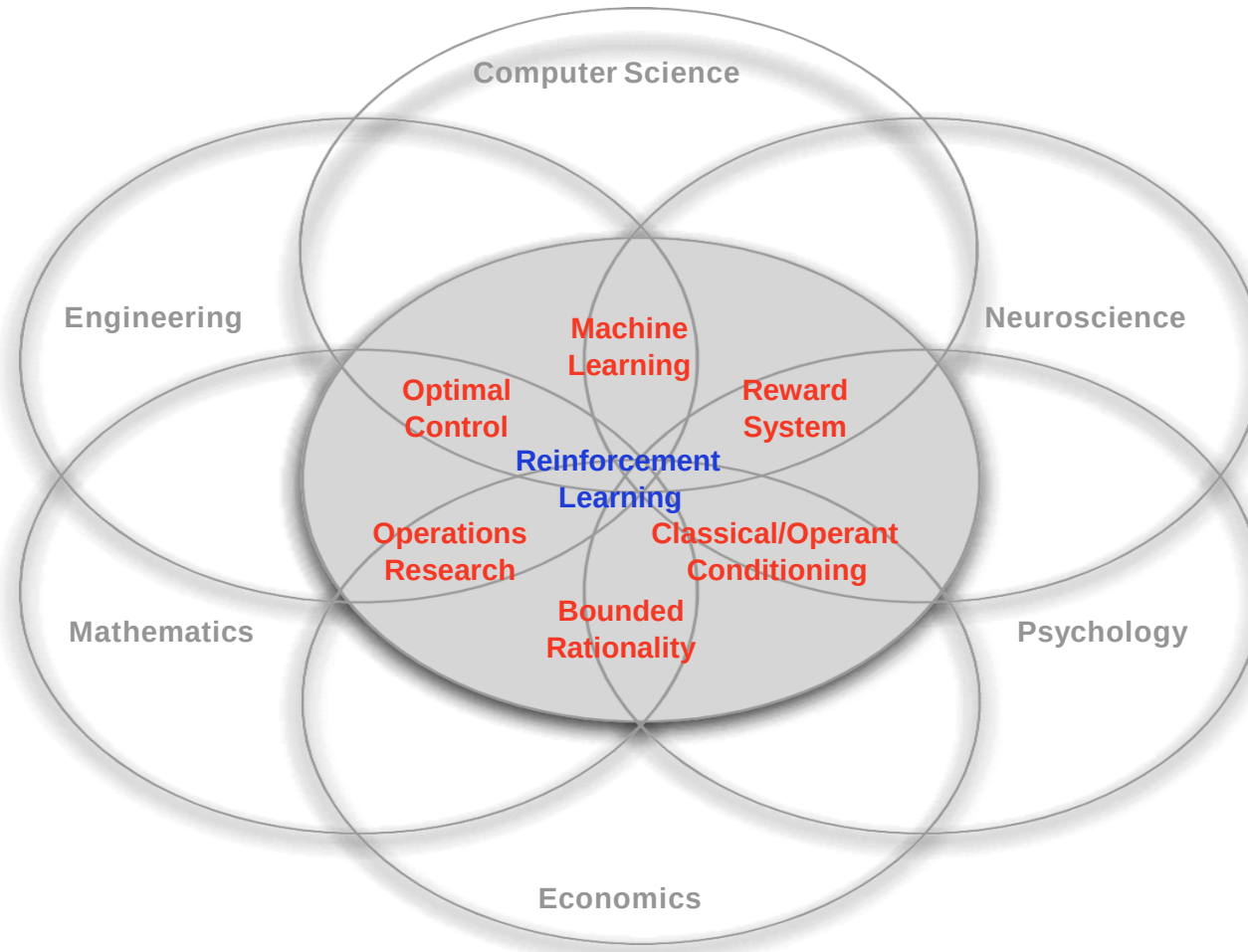
The phenomenon of reinforcement learning

- Learning from Experience Plays a Role in ...



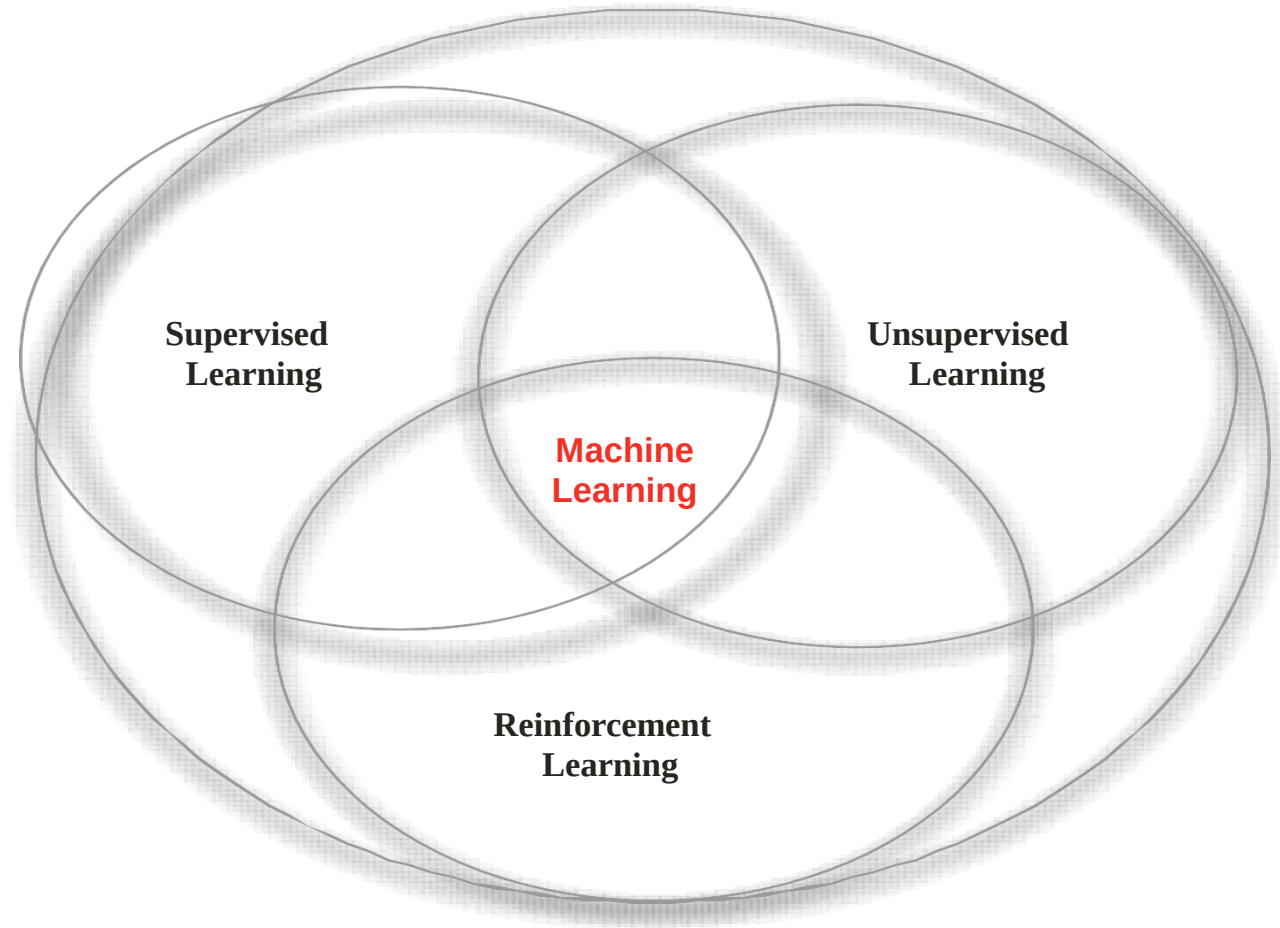
The phenomenon of reinforcement learning

- Learning from Experience Plays a Role in ...



The phenomenon of reinforcement learning

- A branch of Machine Learning



Reinforcement Learning

Reinforcement learning

- Reinforcement learning is the problem faced by an agent that learns behavior through trial-and-error interactions with a dynamic environment.
- Reinforcement Learning is learning how to act in order to maximize a numerical reward.
- RL is not a type of neural network, nor is it an alternative to neural networks. Rather, it is an orthogonal approach for Learning Machine.
- It emphasizes learning feedback that evaluates the learner's performance without providing standards of correctness in the form of behavioral targets.

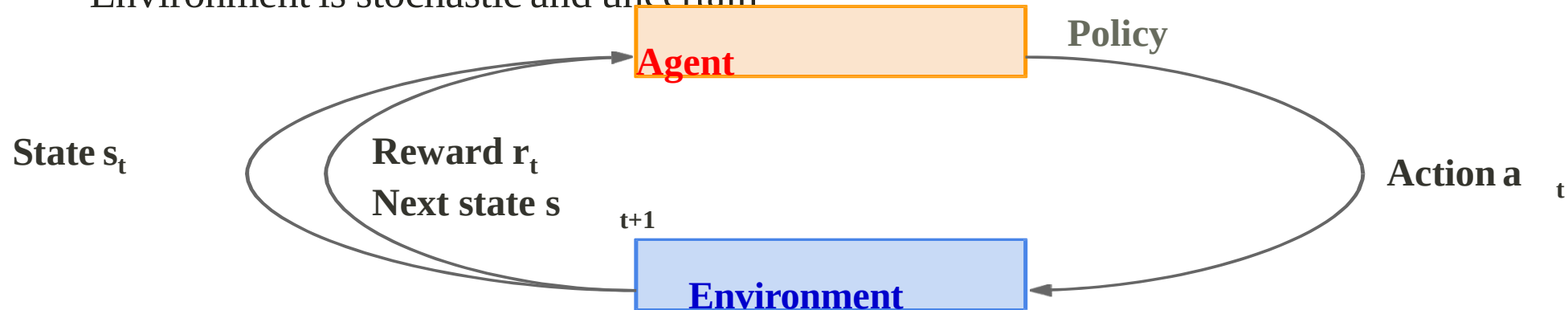
Training Info = evaluations (“rewards” / “penalties”)



Objective: get as much reward as possible

Reinforcement learning

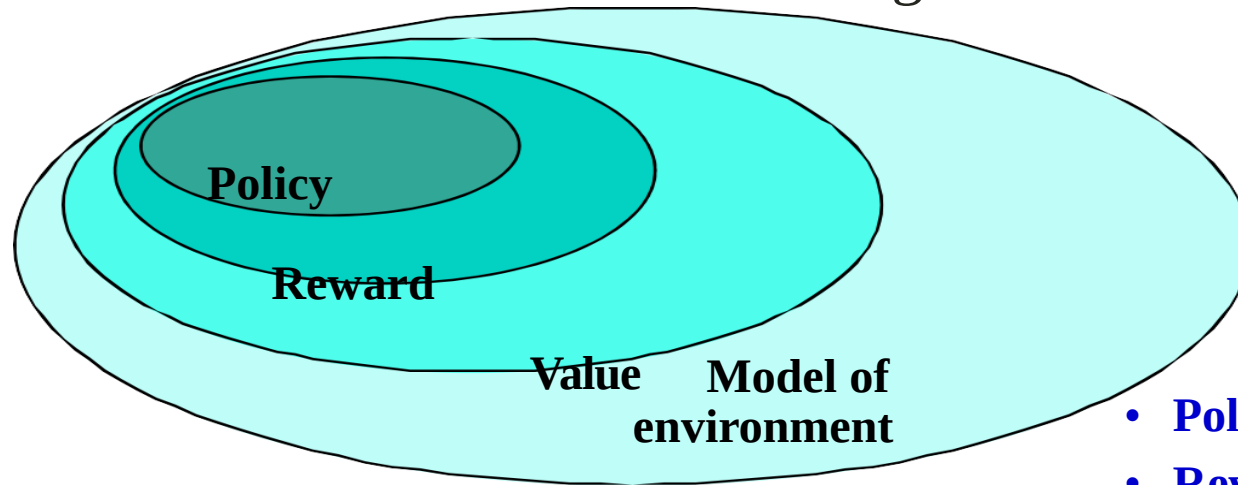
- Elements of reinforcement learning
 - Temporally situated
 - Continual learning and planning
 - Object is to affect the environment
 - Environment is stochastic and uncertain



- **Agent:** Intelligent programs
- **Environment:** External condition
- **Policy:**
 - Defines the agent's behavior at a given time
 - A mapping from states to actions
 - Lookup tables or simple function

Reinforcement learning

- Elements of reinforcement learning



- **Policy:** what to do
- **Reward:** what is good
- **Value:** what is good because it predicts reward
- **Model:** what follows what

- **Reward function:**

- Defines the goal in an RL problem
- Policy is altered to achieve this goal
 - The task is to learn an optimal policy that maps states of the world to actions of the agent. I.e., if this patch of room is dirty, I clean it. If my battery is empty, I recharge it.

Markov Decision Process

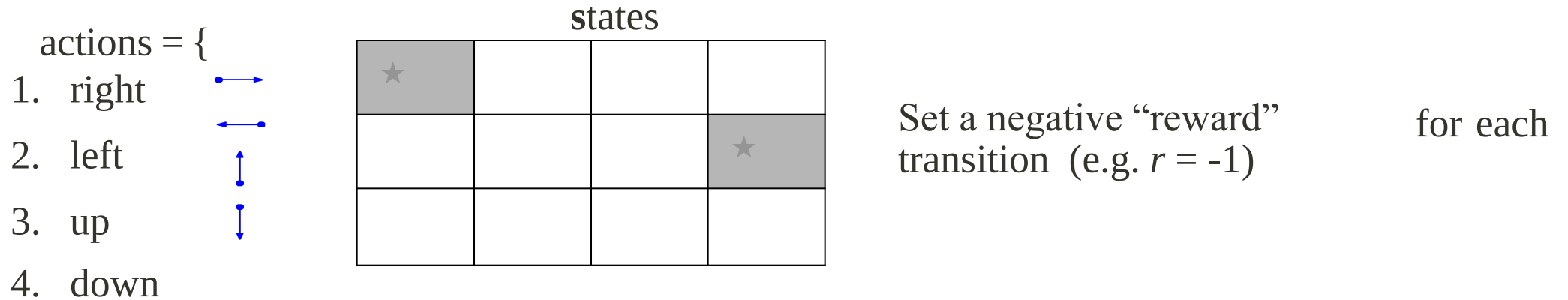
- MDP is a classical formalization of sequential decision making, where actions influence not just immediate rewards, but also subsequent situations, or states, and through those future rewards,
- It meant to be a straightforward framing of the problem of learning from interaction to achieve a goal.
 - The learner and decision maker is called the **agent**. The thing it interacts with, comprising everything outside the agent, is called the **environment**.
 - These interact continually, the agent selecting actions and the environment responding to these actions and presenting **new situations to the agent**.
- Mathematical formulation of the RL problem defined by:
 - $\mathcal{S}$ set of possible states
 - $\mathcal{A}$ set of possible actions
 - $\mathcal{R}$ distribution of reward given (state, action) pair
 - $\mathbb{P}$ transition probability i.e. distribution over next state given (state, action) pair
 - γ discount factor

Markov Decision Process

- At time step $t=0$, environment samples initial state $s_0 \sim p(s_0)$
- Then, for $t=0$ until done:
 - Agent selects action a_t
 - Environment samples reward $r_t \sim R(\cdot | s_t, a_t)$
 - Environment samples next state $s_{t+1} \sim P(\cdot | s_t, a_t)$
 - Agent receives reward r_t and next state s_{t+1}
- A **policy** u is a function from S to A that specifies what action to take in each state
- **Objective:** find policy u^* that maximizes cumulative discounted reward: $\sum_{t \geq 0} \gamma^t r_t$

Markov Decision Process

- A simple MDP: Grid World



- Objective: reach one of terminal states (greyed out) least number of in actions
- **Example: Recycle robot MDP**
 - At each step, robot has to decide whether it should (1) actively search for a can, (2) wait for someone to bring it a can, or (3) go to home base and recharge.
 - Searching is better but runs down the battery; if runs out of power while searching, has to be rescued (which is bad).

Markov Decision Process

- **Example: Recycle robot MDP**

- Decisions made on basis of current energy level: high, low.
- Reward = number of cans collected

$S = \{\text{high}, \text{low}\}$

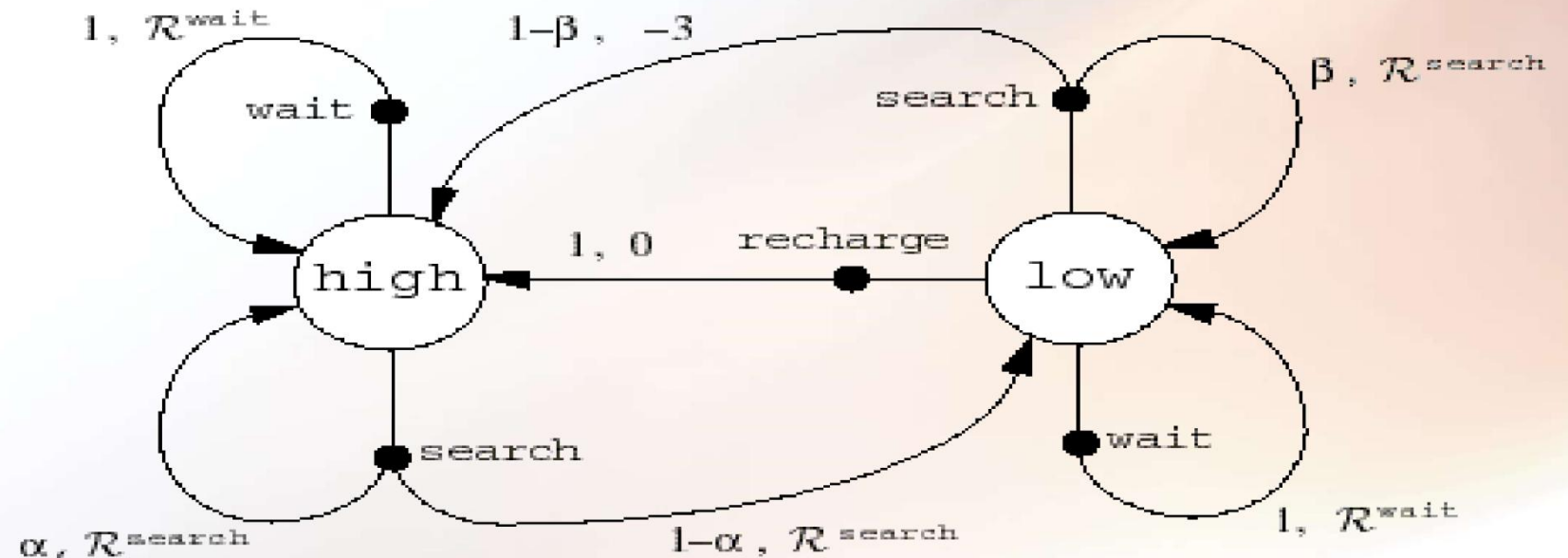
$A(\text{high}) = \{\text{search}, \text{wait}\}$

$A(\text{low}) = \{\text{search}, \text{wait}, \text{recharge}\}$

$R^{\text{search}} = \text{expected no. of cans while searching}$

$R^{\text{wait}} = \text{expected no. of cans while waiting}$

$$R^{\text{search}} > R^{\text{wait}}$$



- Practical examples in Python
- Implement k-means clustering